In [1]:
```
import pandas as pd
import glob
import os
```

Combined all the csv dataframes to one big table

In [2]:
```
all_files = glob.glob("*_daily.csv") ## find all daily csv files in the folder

df_list = [] #empty list to store each files table
for file in all_files:
    symbol = os.path.basename(file).replace("_daily.csv", "") #get symbol name from
    temp_df = pd.read_csv(file) #Read csv file into a tables
    temp_df['Symbol'] = symbol #add symbol columns to every row
    df_list.append(temp_df) #add this table to the list

combined_df = pd.concat(df_list, ignore_index=True) # join all 26 tables into one
combined_df['Date'] = pd.to_datetime(combined_df['Date']) #convert date from text t
combined_df = combined_df.sort_values(['Symbol', 'Date']).reset_index(drop=True) #s

print(combined_df.shape) #show got how many row and columns
print(combined_df['Symbol'].unique()) #show list of all symbol
combined_df.head() #show first 5 rows
```

```
(151282, 6)
['AUDUSD' 'CADUSD' 'CHFUSD' 'CNYUSD' 'DKKUSD' 'EURGBP' 'EURJPY' 'EURUSD'
 'GBPJPY' 'GBPUSD' 'Gold' 'HUFUSD' 'IDRUSD' 'INRUSD' 'JPYUSD' 'KRWUSD'
 'MXNUSD' 'NOKUSD' 'NZDUSD' 'Oil' 'PLNUSD' 'SEKUSD' 'SGDUSD' 'Silver'
 'THBUSD' 'ZARUSD']
```

Out[2]:

	Date	Open	High	Low	Close	Symbol
0	2006-05-15	0.76283	0.76899	0.76069	0.76711	AUDUSD
1	2006-05-16	0.76687	0.77262	0.75758	0.75942	AUDUSD
2	2006-05-17	0.75942	0.76758	0.75873	0.76400	AUDUSD
3	2006-05-18	0.76388	0.76482	0.75301	0.75717	AUDUSD
4	2006-05-21	0.75689	0.75689	0.74710	0.75341	AUDUSD

Step 4 :Calculate MA + ATR per symbol using groupby

In [3]:
```
def add_indicators(group):
    group = group.sort_values('Date').reset_index(drop=True) #sort by date oldest t

    # Moving Average
    group['MA_short'] = group['Close'].rolling(window=10).mean() #average of last 1
    group['MA_long'] = group['Close'].rolling(window=50).mean() #average of last 50

    # ATR (Average True Range)
```

```
    group['prev_close'] = group['Close'].shift(1)   # get previous candle's close p
    group['tr1'] = group['High'] - group['Low']   # candle size (high to low)
    group['tr2'] = (group['High'] - group['prev_close']).abs()   # gap up from prev
    group['tr3'] = (group['Low'] - group['prev_close']).abs()    # gap down from pr
    group['true_range'] = group[['tr1', 'tr2', 'tr3']].max(axis=1)   # biggest of t
    group['ATR'] = group['true_range'].rolling(window=14).mean()   # average True R

    return group   #return the table with new columns added

#apply add_indicators to each symbol seperately
combined_df = combined_df.groupby('Symbol', group_keys=False)[combined_df.columns].
# remove temporary columns, keep only MA_Short , MA long , ATR
combined_df = combined_df.drop(columns=['prev_close', 'tr1', 'tr2', 'tr3', 'true_ra
#show first 5 row to check
combined_df.head()
```

Out[3]:

	Date	Open	High	Low	Close	Symbol	MA_short	MA_long	ATR
0	2006-05-15	0.76283	0.76899	0.76069	0.76711	AUDUSD	NaN	NaN	NaN
1	2006-05-16	0.76687	0.77262	0.75758	0.75942	AUDUSD	NaN	NaN	NaN
2	2006-05-17	0.75942	0.76758	0.75873	0.76400	AUDUSD	NaN	NaN	NaN
3	2006-05-18	0.76388	0.76482	0.75301	0.75717	AUDUSD	NaN	NaN	NaN
4	2006-05-21	0.75689	0.75689	0.74710	0.75341	AUDUSD	NaN	NaN	NaN

Just checking and filter and Eurusd Wheather it can work or not

The reason is it a quick check to make sure Ma and Atr were calculate correctly before moving on the next step.

Why choose EUR USD Specifacally ?

- Is Because it is the most well known pair ,so it easy to tell if the number look right or not

In [33]:

```
eurusd_check = combined_df[combined_df['Symbol'] == 'EURUSD']
eurusd_check[['Date', 'Close', 'MA_short', 'MA_long', 'ATR']].tail(10)  # show last
```

Out[33]:

	Date	Close	MA_short	MA_long	ATR
46023	2025-06-24	1.16122	1.154026	1.137052	0.008637
46024	2025-06-25	1.16837	1.155770	1.137740	0.008957
46025	2025-06-26	1.16922	1.156689	1.138539	0.008964
46026	2025-06-29	1.17275	1.158397	1.139200	0.009005
46027	2025-06-30	1.17872	1.160756	1.139749	0.009132
46028	2025-07-01	1.18055	1.163985	1.140658	0.008201
46029	2025-07-02	1.18002	1.167212	1.141603	0.008009
46030	2025-07-03	1.17715	1.169799	1.142403	0.007678
46031	2025-07-06	1.17808	1.172598	1.143278	0.007745
46032	2025-07-07	1.17365	1.173973	1.143933	0.007939

MA Crossover Signal Logic

In [34]:

```
import numpy as np

def add_signals(group, sl_atr_mult=1.5, rr_ratio=2.0):
    group = group.sort_values('Date').reset_index(drop=True)  # sort by date oldest

    # --- MA Crossover Signal ---
    valid = group['MA_short'].notna() & group['MA_long'].notna()  # both MA must ha
    above = group['MA_short'] > group['MA_long']                  # is MA_short abo
    prev_above = above.shift(1, fill_value=False)                 # was MA_short ab
    prev_valid = valid.shift(1, fill_value=False)                 # were both MA va
    cross_ok = valid & prev_valid                                  # both today ana

    # Signal: 1 = Buy, -1 = Sell, 0 = no trade
    group['Signal'] = 0
    group.loc[cross_ok & above & ~prev_above, 'Signal'] = 1   # MA_short crossed UP
    group.loc[cross_ok & ~above & prev_above, 'Signal'] = -1  # MA_short crossed DO

    # --- Stop Loss and Take Profit using ATR ---
    sl_dist = group['ATR'] * sl_atr_mult          # stop loss distance = ATR × 1.5

    buy  = group['Signal'] == 1                   # rows where signal is Buy
    sell = group['Signal'] == -1                  # rows where signal is Sell

    group['Entry'] = np.where(buy | sell, group['Close'], np.nan)  # entry price =

    group['SL'] = np.where(buy,  group['Close'] - sl_dist,   # Buy  → SL below entr
                  np.where(sell, group['Close'] + sl_dist,    # Sell → SL above ent
                  np.nan))                                     # no signal → no SL

    group['TP'] = np.where(buy,  group['Close'] + sl_dist * rr_ratio,  # Buy  → TP
                  np.where(sell, group['Close'] - sl_dist * rr_ratio,  # Sell → TP
```

```
                    np.nan))                                                    # no signal
    return group  # return table with Signal, Entry, SL, TP columns added
```

Cell 2: Apply per symbol (same style as your Step 4)

In [64]:
```
#run add_signals for each symbol one by one
combined_df = combined_df.groupby('Symbol', group_keys=False)[combined_df.columns].
combined_df = combined_df.reset_index(drop=True) # restart row numbers from 0

print(combined_df['Signal'].value_counts()) # show how many Buy , Sell and no trade
```

```
Signal
 0    147197
 1      2046
-1      2039
Name: count, dtype: int64
```

Check Eurusd signals

In [63]:
```
#filter EURUSD rows that have a Buy or Sell signal only (remove No trade rows)
eurusd_signals = combined_df[(combined_df['Symbol'] == 'EURUSD') & (combined_df['Si
# show last 10 signals with these columns only
eurusd_signals[['Date', 'Close', 'MA_short', 'MA_long', 'Signal', 'SL', 'TP']].tail
```

Out[63]:

	Date	Close	MA_short	MA_long	Signal	SL	TP
45540	2023-08-14	1.09099	1.096821	1.096848	-1	1.103983	1.065004
45602	2023-11-09	1.07104	1.063875	1.062918	1	1.057696	1.097727
45655	2024-01-23	1.08802	1.091967	1.092471	-1	1.097332	1.069396
45689	2024-03-11	1.09430	1.086860	1.085752	1	1.086212	1.110476
45705	2024-04-01	1.07372	1.082788	1.083325	-1	1.082960	1.055240
45738	2024-05-16	1.08678	1.079443	1.078574	1	1.078059	1.104223
45762	2024-06-19	1.07515	1.076665	1.076966	-1	1.086340	1.052770
45778	2024-07-11	1.08728	1.080027	1.079567	1	1.079199	1.103441
45842	2024-10-09	1.09433	1.105008	1.105602	-1	1.105352	1.072286
45934	2025-02-18	1.04854	1.039656	1.039051	1	1.035299	1.075021

In [37]:
```
## Backtest Win or Loss per trade
## Cell 1 Backtest Functions
def backtest(group):
    group = group.reset_index(drop=True)            # clean 0,1,2 index
    trades = []                                      # store each trade here

    for i in group.index[group['Signal'] != 0]:      # loop signal rows only
        d = group.at[i, 'Signal']                    # 1 = buy, -1 = sell
```

```
        entry = group.at[i, 'Entry']                    # entry price
        sl = group.at[i, 'SL']                          # stop loss
        tp = group.at[i, 'TP']                          # take profit
        rr = abs(tp - entry) / abs(sl - entry)          # reward : risk

        for j in range(i + 1, len(group)):              # check future candles
            high = group.at[j, 'High']                  # candle high
            low = group.at[j, 'Low']                    # candle low

            hit_sl = low <= sl if d == 1 else high >= sl   # SL touched?
            hit_tp = high >= tp if d == 1 else low <= tp   # TP touched?

            if hit_sl or hit_tp:                        # trade closed
                win = hit_tp and not hit_sl             # both same day = loss (safe)
                trades.append({
                    'Symbol': group.at[i, 'Symbol'], #Forex trading pair
                    'Entry_Date': group.at[i, 'Date'],  # You can manually input En
                    'Exit_Date': group.at[j, 'Date'],   # You can manually input Ex
                    'Direction': 'Buy' if d == 1 else 'Sell',  # You having a optio
                    'Entry': entry,     #Place to entry Trade
                    'Exit': tp if win else sl,  #When it get Tp or Sl it will auto
                    'Result': 'Win' if win else 'Loss',  #When it hit tp it mean wi
                    'R': rr if win else -1              # Risk reward ratio like 1:1
                })
                break                                   # stop, go to next trade
    return pd.DataFrame(trades)                         # table of trades
```

Cell 2 Run for all Symbol

In [38]:
```
trades_df = pd.concat(
    [backtest(g) for _, g in combined_df.groupby('Symbol')],  # backtest each symbo
    ignore_index=True                                          # join into one tabl
)
print(trades_df.shape)                    # (trades, columns)
print(trades_df['Result'].value_counts()) # wins vs losses
trades_df.head()
```

```
C:\Users\USER\AppData\Local\Temp\ipykernel_34392\2561286359.py:12: RuntimeWarning: i
nvalid value encountered in scalar divide
  rr = abs(tp - entry) / abs(sl - entry)        # reward : risk
(4072, 8)
Result
Loss    2681
Win     1391
Name: count, dtype: int64
```

Out[38]:

	Symbol	Entry_Date	Exit_Date	Direction	Entry	Exit	Result	R
0	AUDUSD	2006-09-14	2006-10-18	Sell	0.75228	0.759972	Loss	-1.0
1	AUDUSD	2006-10-24	2006-10-31	Buy	0.76167	0.774669	Win	2.0
2	AUDUSD	2007-03-05	2007-03-12	Buy	0.76970	0.785581	Win	2.0
3	AUDUSD	2007-03-09	2007-03-15	Sell	0.78131	0.789258	Loss	-1.0
4	AUDUSD	2007-03-19	2007-04-09	Buy	0.80141	0.820747	Win	2.0

Quick Eur Usd checking

In [39]:
```
eu = trades_df[trades_df['Symbol'] == 'EURUSD']           # EURUSD trades
print('Trades:', len(eu))                                  # total trades
print('Win rate:', round((eu['Result'] == 'Win').mean() * 100, 1), '%')  # You can
print('Total R:', eu['R'].sum())                           # Total profit and loss
```

```
Trades: 152
Win rate: 35.5 %
Total R: 9.999999999999824
```

Metrics Function for Cell 1

In [40]:
```
def get_metrics(t):
    wins = t[t['Result'] == 'Win']            # winning trades
    losses = t[t['Result'] == 'Loss']         # losing trades
    gross_win = wins['R'].sum()               # After you risk total you win how mu
    gross_loss = abs(losses['R'].sum())       # After you risk total you loss how m

    return pd.Series({
        'Trades': len(t),                                          # Total trades
        'Win_Rate_%': round(len(wins) / len(t) * 100, 1),          # How much my wi
        'Total_R': round(t['R'].sum(), 1),                         # Total risk ana
        ##won / lost is after divided the number is above one then it mean profitab
        'Profit_Factor': round(gross_win / gross_loss, 2) if gross_loss > 0 else np
        'Expectancy_R': round(t['R'].mean(), 2)                    # Average risk p
    })
```

Cell 2 Metrics per second

In [65]:
```
summary = trades_df.groupby('Symbol')[['Result', 'R']].apply(get_metrics)  # calcul
summary.sort_values('Total_R', ascending=False)                            # check
```

Out[65]:

Symbol	Trades	Win_Rate_%	Total_R	Profit_Factor	Expectancy_R
Silver	177.0	41.2	42.0	1.40	0.24
Oil	181.0	39.8	35.0	1.32	0.19
CNYUSD	146.0	38.4	22.0	1.24	0.15
THBUSD	123.0	39.0	21.0	1.28	0.17
INRUSD	146.0	37.7	19.0	1.21	0.13
DKKUSD	146.0	37.7	19.0	1.21	0.13
EURJPY	155.0	37.4	19.0	1.20	0.12
Gold	159.0	37.1	18.0	1.18	0.11
PLNUSD	153.0	36.6	15.0	1.15	0.10
NZDUSD	147.0	36.7	15.0	1.16	0.10
NOKUSD	166.0	35.5	11.0	1.10	0.07
AUDUSD	136.0	36.0	11.0	1.13	0.08
JPYUSD	194.0	35.1	10.0	1.08	0.05
EURUSD	152.0	35.5	10.0	1.10	0.07
CHFUSD	158.0	35.4	10.0	1.10	0.06
GBPUSD	150.0	35.3	9.0	1.09	0.06
MXNUSD	179.0	34.6	7.0	1.06	0.04
HUFUSD	156.0	34.0	3.0	1.03	0.02
SGDUSD	130.0	33.1	-1.0	0.99	-0.01
EURGBP	204.0	31.9	-9.0	0.94	-0.04
CADUSD	160.0	28.7	-22.0	0.81	-0.14
GBPJPY	152.0	28.3	-23.0	0.79	-0.15
SEKUSD	165.0	28.5	-24.0	0.80	-0.15
ZARUSD	179.0	27.9	-29.0	0.78	-0.16
KRWUSD	176.0	26.7	-35.0	0.73	-0.20
IDRUSD	82.0	12.2	-52.0	0.28	-0.63

Cell 3 Overall metrics (all symbols)

In [70]:
```
# overall metrics across all symbols combined
overall = get_metrics(trades_df[['Result', 'R']])
pd.DataFrame([overall])
```

Out[70]:

	Trades	Win_Rate_%	Total_R	Profit_Factor	Expectancy_R
0	4072.0	34.2	101.0	1.04	0.02

In [ ]:
```
-Total Trades
-Total Win Accurate
- Total amount you risk
- Profit factor is above 1 than is profitable
-Expectancy it mean the average profit you win per trade like if is 0.02 is mean yo
```

How to read the results:

Metric Good sign Win Rate Above 33.3%. With RR 1:2, that's the break-even point. Total R Positive Profit Factor Above 1.0. It's the R won divided by the R lost. Expectancy Above 0. It's the average R you make per trade.

Step 8 Account Growth

In [71]:
```
def account_growth(trades, account_size=1000, risk_pct=1.0):
    t = trades.sort_values('Exit_Date').reset_index(drop=True)  # order hit tp or s
    balance = account_size                                      # starting money
    pnls, balances = [], []                                     # check backtesting

    for r in t['R']:                                            # loop each trade's
        risk_amount = balance * risk_pct / 100                  # risk per trade
        pnl = r * risk_amount                                   # R  mean dollar ,h
        balance += pnl                                          # update balance
        pnls.append(pnl)
        balances.append(balance)

    t['PnL_$'] = pnls                                           # profit/loss per t
    t['Balance'] = balances                                     # total you earn ho
    return t
```

Cell 2 Run on EurUsd

In [72]:
```
eu_growth = account_growth(trades_df[trades_df['Symbol'] == 'EURUSD'],
                           account_size=1000, risk_pct=1.0)     # $1000 as a defaul

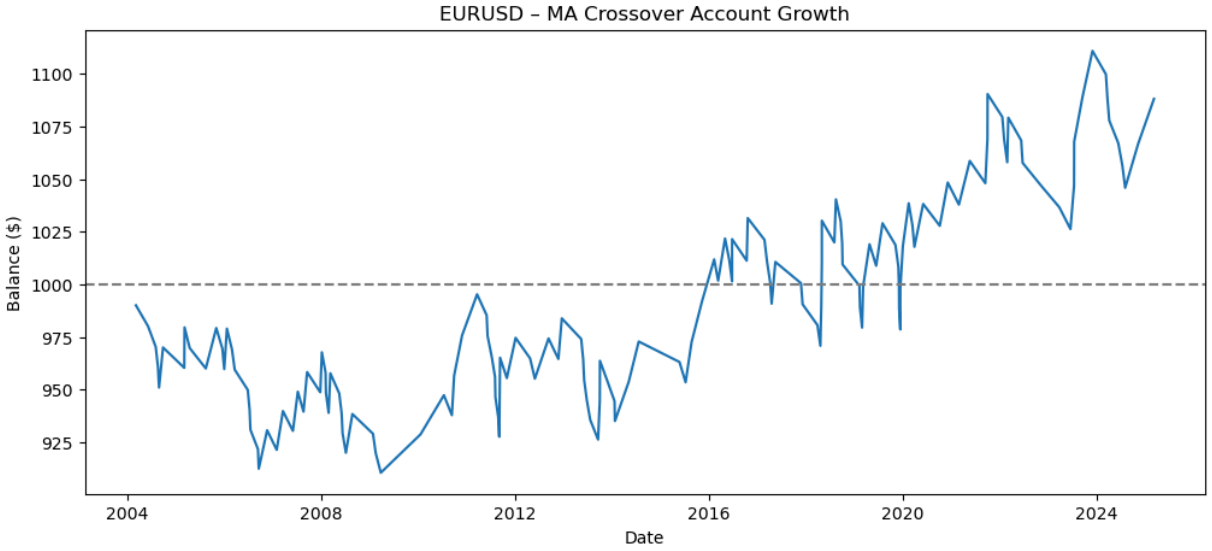
print('Final balance: $', round(eu_growth['Balance'].iloc[-1], 2))   # After backte
print('Avg win: $', round(eu_growth[eu_growth['PnL_$'] > 0]['PnL_$'].mean(), 2))  #
print('Avg loss: $', round(eu_growth[eu_growth['PnL_$'] < 0]['PnL_$'].mean(), 2)) #
```

```
Final balance: $ 1088.07
Avg win: $ 19.65
Avg loss: $ -9.93
```

Cell 3 Equity Curve Chart

In [45]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5)) ## the size of width and height
plt.plot(eu_growth['Exit_Date'], eu_growth['Balance'])          # balance over time
plt.axhline(1000, color='gray', linestyle='--')                 # starting balances
plt.title('EURUSD – MA Crossover Account Growth')
plt.xlabel('Date')
plt.ylabel('Balance ($)')
plt.show()
```



Cell 4 Daily P/L

In [73]:
```
daily_pnl = eu_growth.groupby('Exit_Date')['PnL_$'].sum()       # total earn per da
daily_pnl.tail(round(10))                                              # last 10 tr
```

Out[73]:
```
Exit_Date
2023-09-14    21.353865
2023-11-28    21.780942
2024-03-08   -11.108280
2024-03-19   -10.997198
2024-04-02   -10.887226
2024-06-09   -10.778353
2024-07-10   -10.670570
2024-07-31   -10.563864
2024-11-06    20.916451
2025-03-05    21.334780
Name: PnL_$, dtype: float64
```

Step 9 Add the other 4 strategies

Cell Helper Function

In [47]:
```
def cross_signal(fast, slow):                              # crossover logic (MA + EMA)
    valid = fast.notna() & slow.notna()                    # both lines have values
    above = fast > slow                                    # fast above slow?
    prev_above = above.shift(1, fill_value=False)          # yesterday
    prev_valid = valid.shift(1, fill_value=False)
    ok = valid & prev_valid
    sig = pd.Series(0, index=fast.index)                   # default 0 Series it mean
    sig[ok & above & ~prev_above] = 1                      # cross up = BUY
    sig[ok & ~above & prev_above] = -1                     # cross down = SELL
    return sig

def first_only(sig):                                       # keep only first day of a s
    return sig.where(sig != sig.shift(1), 0)               # repeated days → 0

def add_sl_tp(group, sl_atr_mult=1.5, rr_ratio=2.0):   # same SL/TP logic as Step 5
    sl_dist = group['ATR'] * sl_atr_mult    # stop loss distance = ATR  x 1.5
    buy, sell = group['Signal'] == 1, group['Signal'] == -1  #True where signal is
    group['Entry'] = np.where(buy | sell, group['Close'], np.nan)  #entry price = c
    group['SL'] = np.where(buy, group['Close'] - sl_dist, #When it buy the sl will
                  np.where(sell, group['Close'] + sl_dist, np.nan)) #When it sell t
    group['TP'] = np.where(buy, group['Close'] + sl_dist * rr_ratio, #Buy -> the TP
                  np.where(sell, group['Close'] - sl_dist * rr_ratio, np.nan)) #Sel
    return group
```

Cell 2 the 5 strategy

In [48]:
```
# 1. MA Crossover – simple average cross
def strat_ma(g, fast=10, slow=50):
    return cross_signal(g['Close'].rolling(fast).mean(),
                        g['Close'].rolling(slow).mean())

# 2. EMA Crossover – same idea, but recent prices weigh more (reacts faster)
def strat_ema(g, fast=12, slow=26):
    return cross_signal(g['Close'].ewm(span=fast, adjust=False).mean(),
                        g['Close'].ewm(span=slow, adjust=False).mean())

# 3. Support & Resistance – bounce off recent low/high
def strat_sr(g, lookback=20):
    support = g['Low'].rolling(lookback).min().shift(1)       # lowest low (past 20
    resistance = g['High'].rolling(lookback).max().shift(1)   # highest high (past
    zone = g['ATR'] * 0.5                                     # "near" = within 0.5
    buy = (g['Low'] <= support + zone) & (g['Close'] > support)          # touch su
    sell = (g['High'] >= resistance - zone) & (g['Close'] < resistance)  # touch re
    sig = pd.Series(np.where(buy & ~sell, 1, np.where(sell & ~buy, -1, 0)), index=g
    return first_only(sig)

# 4. Breakout – close breaks past the recent high/low
def strat_breakout(g, lookback=20):
    high_n = g['High'].rolling(lookback).max().shift(1)       # past 20-day high
    low_n = g['Low'].rolling(lookback).min().shift(1)         # past 20-day low
    sig = pd.Series(np.where(g['Close'] > high_n, 1,          # break up = BUY
                    np.where(g['Close'] < low_n, -1, 0)),     # break down = SELL
                    index=g.index)
```

```
    return first_only(sig)

# 5. Fibonacci Retracement – buy the pullback in trend at 61.8%
def strat_fib(g, lookback=50, level=0.618):
    swing_high = g['High'].rolling(lookback).max().shift(1)   # recent swing high
    swing_low = g['Low'].rolling(lookback).min().shift(1)     # recent swing low
    rng = swing_high - swing_low                              # swing size
    uptrend = g['Close'] > g['Close'].rolling(lookback).mean()  # trend filter
    buy_lvl = swing_high - rng * level                        # 61.8% pullback (upt
    sell_lvl = swing_low + rng * level                        # 61.8% pullback (dow
    buy = uptrend & (g['Low'] <= buy_lvl) & (g['Close'] > buy_lvl)       # touch +
    sell = ~uptrend & (g['High'] >= sell_lvl) & (g['Close'] < sell_lvl)  # touch +
    sig = pd.Series(np.where(buy, 1, np.where(sell, -1, 0)), index=g.index)
    return first_only(sig)
```

Cell 3 One function to run any strategy

In [49]:
```
STRATEGIES = {                                  # name → function (dashboard dropd
    'MA Crossover': strat_ma,
    'EMA Crossover': strat_ema,
    'Support & Resistance': strat_sr,
    'Breakout': strat_breakout,
    'Fibonacci': strat_fib
}

def add_strategy(group, strategy, sl_atr_mult=1.5, rr_ratio=2.0):
    group = group.sort_values('Date').reset_index(drop=True)
    group['Signal'] = STRATEGIES[strategy](group).values      # pick strategy → sig
    return add_sl_tp(group, sl_atr_mult, rr_ratio)            # add Entry/SL/TP

def run_strategy(df, strategy, sl_atr_mult=1.5, rr_ratio=2.0):
    signals = df.groupby('Symbol', group_keys=False)[df.columns].apply(
        lambda g: add_strategy(g, strategy, sl_atr_mult, rr_ratio))           # sig
    trades = pd.concat([backtest(g) for _, g in signals.groupby('Symbol')],
                       ignore_index=True)                                    # back
    trades['Strategy'] = strategy                                            # labe
    return trades
```

Cell 4 Compare all 5 strategies

In [50]:
```
results = []
for name in STRATEGIES:                         # loop each strategy
    t = run_strategy(combined_df, name)         # get trades
    m = get_metrics(t)                          # Step 7 metrics
    m['Strategy'] = name
    results.append(m)
    print(name, 'done')                         # progress

comparison = pd.DataFrame(results).set_index('Strategy')
comparison.sort_values('Total_R', ascending=False)   # best strategy on top
```

```
C:\Users\USER\AppData\Local\Temp\ipykernel_34392\2561286359.py:12: RuntimeWarning: i
nvalid value encountered in scalar divide
  rr = abs(tp - entry) / abs(sl - entry)        # reward : risk
MA Crossover done
C:\Users\USER\AppData\Local\Temp\ipykernel_34392\2561286359.py:12: RuntimeWarning: i
nvalid value encountered in scalar divide
  rr = abs(tp - entry) / abs(sl - entry)        # reward : risk
EMA Crossover done
C:\Users\USER\AppData\Local\Temp\ipykernel_34392\2561286359.py:12: RuntimeWarning: i
nvalid value encountered in scalar divide
  rr = abs(tp - entry) / abs(sl - entry)        # reward : risk
Support & Resistance done
C:\Users\USER\AppData\Local\Temp\ipykernel_34392\2561286359.py:12: RuntimeWarning: i
nvalid value encountered in scalar divide
  rr = abs(tp - entry) / abs(sl - entry)        # reward : risk
Breakout done
C:\Users\USER\AppData\Local\Temp\ipykernel_34392\2561286359.py:12: RuntimeWarning: i
nvalid value encountered in scalar divide
  rr = abs(tp - entry) / abs(sl - entry)        # reward : risk
Fibonacci done
```

Out[50]:

Strategy	Trades	Win_Rate_%	Total_R	Profit_Factor	Expectancy_R
Breakout	8522.0	34.0	163.0	1.03	0.02
MA Crossover	4072.0	34.2	101.0	1.04	0.02
EMA Crossover	5244.0	33.6	36.0	1.01	0.01
Fibonacci	5120.0	24.7	-1331.0	0.65	-0.26
Support & Resistance	23099.0	28.8	-3152.0	0.81	-0.14

Step 10 Dashboard

In [51]:
```
%pip install ipywidgets
```

```
Requirement already satisfied: ipywidgets in c:\users\user\anaconda3\lib\site-packag
es (8.1.7)
Requirement already satisfied: comm>=0.1.3 in c:\users\user\anaconda3\lib\site-packa
ges (from ipywidgets) (0.2.3)
Requirement already satisfied: ipython>=6.1.0 in c:\users\user\anaconda3\lib\site-pa
ckages (from ipywidgets) (9.7.0)
Requirement already satisfied: traitlets>=4.3.1 in c:\users\user\anaconda3\lib\site-
packages (from ipywidgets) (5.14.3)
Requirement already satisfied: widgetsnbextension~=4.0.14 in c:\users\user\anaconda3
\lib\site-packages (from ipywidgets) (4.0.14)
Requirement already satisfied: jupyterlab_widgets~=3.0.15 in c:\users\user\anaconda3
\lib\site-packages (from ipywidgets) (3.0.15)
Requirement already satisfied: colorama>=0.4.4 in c:\users\user\anaconda3\lib\site-p
ackages (from ipython>=6.1.0->ipywidgets) (0.4.6)
Requirement already satisfied: decorator>=4.3.2 in c:\users\user\anaconda3\lib\site-
packages (from ipython>=6.1.0->ipywidgets) (5.2.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in c:\users\user\anaco
nda3\lib\site-packages (from ipython>=6.1.0->ipywidgets) (1.1.1)
Requirement already satisfied: jedi>=0.18.1 in c:\users\user\anaconda3\lib\site-pack
ages (from ipython>=6.1.0->ipywidgets) (0.19.2)
Requirement already satisfied: matplotlib-inline>=0.1.5 in c:\users\user\anaconda3\l
ib\site-packages (from ipython>=6.1.0->ipywidgets) (0.2.1)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in c:\users\user\anacon
da3\lib\site-packages (from ipython>=6.1.0->ipywidgets) (3.0.52)
Requirement already satisfied: pygments>=2.11.0 in c:\users\user\anaconda3\lib\site-
packages (from ipython>=6.1.0->ipywidgets) (2.19.2)
Requirement already satisfied: stack_data>=0.6.0 in c:\users\user\anaconda3\lib\site
-packages (from ipython>=6.1.0->ipywidgets) (0.6.3)
Requirement already satisfied: wcwidth in c:\users\user\anaconda3\lib\site-packages
(from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0->ipywidgets) (0.2.13)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in c:\users\user\anaconda3\lib\si
te-packages (from jedi>=0.18.1->ipython>=6.1.0->ipywidgets) (0.8.5)
Requirement already satisfied: executing>=1.2.0 in c:\users\user\anaconda3\lib\site-
packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in c:\users\user\anaconda3\lib\site-
packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (3.0.0)
Requirement already satisfied: pure_eval in c:\users\user\anaconda3\lib\site-package
s (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (0.2.3)
Note: you may need to restart the kernel to use updated packages.
```

Cell 1 from Step 10b to check your intraday files

In [52]:
```
import glob
```

In [53]:
```
files = glob.glob('forex_clean_data/**/*.csv' , recursive=True) #check all CSV File
[f for f in files if '60m' in f or '4h' in f] #filter 60m and 4h in the name
```

Out[53]:
```
[]
```

Cell 1: Check the 4h and 60m files exist before using them

Cell 2: Check the data inside is usable (dates, columns)

In [54]:
```
import os ## operating system  manage your python modules to let your code work wit
import pandas as pd
```

In [55]:
```
files = glob.glob('**/*.csv', recursive=True) #find all CSV files inside forex_clea
print(len(files))
[f for f in files if '60m' in f or '4h' in f][:10] #filter all the 60min and 4hour
```

```
52
```

Out[55]:
```
[]
```

In [56]:
```
files[:10]
```

Out[56]:
```
['AUDUSD_daily.csv',
 'AUDUSD_weekly.csv',
 'CADUSD_daily.csv',
 'CADUSD_weekly.csv',
 'CHFUSD_daily.csv',
 'CHFUSD_weekly.csv',
 'CNYUSD_daily.csv',
 'CNYUSD_weekly.csv',
 'DKKUSD_daily.csv',
 'DKKUSD_weekly.csv']
```

In [57]:
```
os.listdir()
```

Out[57]:
```
['.ipynb_checkpoints',
 'AUDUSD_daily.csv',
 'AUDUSD_weekly.csv',
 'CADUSD_daily.csv',
 'CADUSD_weekly.csv',
 'CHFUSD_daily.csv',
 'CHFUSD_weekly.csv',
 'CNYUSD_daily.csv',
 'CNYUSD_weekly.csv',
 'DKKUSD_daily.csv',
 'DKKUSD_weekly.csv',
 'EURGBP_daily.csv',
 'EURGBP_weekly.csv',
 'EURJPY_daily.csv',
 'EURJPY_weekly.csv',
 'EURUSD_daily.csv',
 'EURUSD_weekly.csv',
 'Forex_Project.ipynb',
 'GBPJPY_daily.csv',
 'GBPJPY_weekly.csv',
 'GBPUSD_daily.csv',
 'GBPUSD_weekly.csv',
 'Gold_daily.csv',
 'Gold_weekly.csv',
 'HUFUSD_daily.csv',
 'HUFUSD_weekly.csv',
 'IDRUSD_daily.csv',
 'IDRUSD_weekly.csv',
 'INRUSD_daily.csv',
 'INRUSD_weekly.csv',
 'instruments.json',
 'JPYUSD_daily.csv',
 'JPYUSD_weekly.csv',
 'KRWUSD_daily.csv',
 'KRWUSD_weekly.csv',
 'MXNUSD_daily.csv',
 'MXNUSD_weekly.csv',
 'My Forex Limitation.ipynb',
 'NOKUSD_daily.csv',
 'NOKUSD_weekly.csv',
 'NZDUSD_daily.csv',
 'NZDUSD_weekly.csv',
 'Oil_daily.csv',
 'Oil_weekly.csv',
 'PLNUSD_daily.csv',
 'PLNUSD_weekly.csv',
 'SEKUSD_daily.csv',
 'SEKUSD_weekly.csv',
 'SGDUSD_daily.csv',
 'SGDUSD_weekly.csv',
 'Silver_daily.csv',
 'Silver_weekly.csv',
 'THBUSD_daily.csv',
 'THBUSD_weekly.csv',
 'ZARUSD_daily.csv',
 'ZARUSD_weekly.csv']
```

In [58]:
```
#check 3 folder one by one
for place in [r'C:\Users\USER\Downloads', r'C:\Users\USER\Desktop', r'C:\Users\USER
    #find all files with '4h' in the name
    found = glob.glob(place + r'\**\*4h*.csv', recursive=True)
    #show folder name and how many files found
    print(place, '→', len(found))
    #show first 3 file paths
    print(found[:3])
```

```
C:\Users\USER\Downloads → 0
[]
C:\Users\USER\Desktop → 0
[]
C:\Users\USER → 0
[]
```

Cell 1: Load the weekly data

In [74]:
```
WEEKLY_TAG = 'weekly'      # change to match your weekly file names

def load_weekly():
    frames = []  #empty list to store each file
    for f in glob.glob(f'**/*{WEEKLY_TAG}*.csv', recursive=True): #find all the csv
        df = pd.read_csv(f, parse_dates=['Date']) #read the files
        df['Symbol'] = os.path.basename(f).split('_')[0]    # get symbol name from
        frames.append(df[['Date', 'Open', 'High', 'Low', 'Close', 'Symbol']]) #keep
    return pd.concat(frames).sort_values(['Symbol', 'Date']).reset_index(drop=True)

DATA = {
    'Daily': combined_df,      # store daily data
    'Weekly': load_weekly(),   #store weekly data
}
for k, v in DATA.items():
    print(k, len(v), 'rows |', v['Date'].min(), '→', v['Date'].max()) #show timefra
```

```
Daily 151282 rows | 2000-01-03 00:00:00 → 2025-07-08 00:00:00
Weekly 30322 rows | 2000-01-06 00:00:00 → 2025-07-08 00:00:00
```

In [60]:
```
# ----- Helper: calculate ATR (for Weekly data) -----
def add_atr(df, n=14):
    df = df.copy()
    prev_close = df['Close'].shift(1)                    # previous candle's close
    tr = pd.concat([df['High'] - df['Low'],              # candle range
                    (df['High'] - prev_close).abs(),     # gap up
                    (df['Low'] - prev_close).abs()],     # gap down
                   axis=1).max(axis=1)                   # True Range
    df['ATR'] = tr.rolling(n).mean()                     # average over 14 candles
    return df


# ----- Dashboard -----
def run_dashboard(timeframe, symbol, strategy, account_size, risk_pct, rr_ratio,
                  sl_atr_mult, start, end, trade_option):
```

```
    df = DATA[timeframe]                                          # Daily or Weekl
      # NEW: keep dates inside the data range
    data_start = df['Date'].min().date()                          # Daily: 2000-01
    data_end = df['Date'].max().date()                            # both: 2025-07-
    if start < data_start or end > data_end:
        print(f"Note: {timeframe} data only covers {data_start} → {data_end}. Dates
    start = max(start, data_start)                                # not earlier th
    end = min(end, data_end)                                      # not later than
    if start > end:
        print('Start date is after end date'); return
    data = df[df['Symbol'] == symbol]                             # one symbol onl
    if 'ATR' not in data.columns:                                 # NEW: Weekly ha
        data = add_atr(data)                                      # NEW: calculate
    data = add_strategy(data, strategy, sl_atr_mult, rr_ratio)    # signals + SL/T
    trades = backtest(data)                                       # win/loss per t

    if trades.empty:
        print('No trades found'); return

    trades = trades[(trades['Entry_Date'] >= pd.Timestamp(start)) &
                    (trades['Entry_Date'] <= pd.Timestamp(end))]   # date range fil
    if trade_option != 'Both':
        trades = trades[trades['Direction'] == trade_option]      # Buy only / Sel

    if trades.empty:
        print('No trades in this range'); return #if do have opportunity to trade i

    m = get_metrics(trades)
    g = account_growth(trades, account_size, risk_pct)
    avg_win = g.loc[g['PnL_$'] > 0, 'PnL_$'].mean()  #it show the average of win
    avg_loss = g.loc[g['PnL_$'] < 0, 'PnL_$'].mean() #it show the average of loss
    daily = g.groupby('Exit_Date')['PnL_$'].sum()  #it will count you total trades

    # ----- Output numbers -----
    print(f"Time frame:      {timeframe}")
    print(f"Trades:          {int(m['Trades'])}")
    print(f"Win rate:        {m['Win_Rate_%']}%")
    print(f"Profit factor:   {m['Profit_Factor']}")
    print(f"Avg win:         ${avg_win:,.2f}")
    print(f"Avg loss:        ${avg_loss:,.2f}")
    print(f"Final balance:   ${g['Balance'].iloc[-1]:,.2f}")

    # ----- Output charts -----
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

    ax1.plot(g['Exit_Date'], g['Balance'])
    ax1.axhline(account_size, color='gray', linestyle='--')
    ax1.set_title(f'{symbol} - {strategy} - {timeframe} - Account Growth')
    ax1.set_ylabel('Balance ($)')

    colors = ['green' if v > 0 else 'red' for v in daily.values]
    ax2.bar(daily.index, daily.values, color=colors, width=5)
    ax2.set_title('Profit / Loss per Trade Day')
    ax2.set_ylabel('P/L ($)')
```

```
    plt.tight_layout()
    plt.show()
```

In [61]:
```
import ipywidgets as widgets
from ipywidgets import interact_manual
import datetime
from IPython.display import display

display(widgets.HTML(                                                    # NEW: info note
    "<b>Data available:</b> Daily 03 Jan 2000 → 08 Jul 2025 | "
    "Weekly 06 Jan 2000 → 08 Jul 2025"))

interact_manual(
    run_dashboard,
    timeframe=widgets.Dropdown(options=['Daily', 'Weekly'], description='Time frame
    symbol=widgets.Dropdown(options=sorted(combined_df['Symbol'].unique()),
                            value='EURUSD', description='Symbol'),
    strategy=widgets.Dropdown(options=list(STRATEGIES.keys()), description='Strateg
    account_size=widgets.FloatText(value=1000, description='Account $'),
    risk_pct=widgets.FloatSlider(value=1, min=0.1, max=5, step=0.1, description='Ri
    rr_ratio=widgets.FloatSlider(value=2, min=0.5, max=5, step=0.5, description='RR
    sl_atr_mult=widgets.FloatSlider(value=1.5, min=0.5, max=4, step=0.5, descriptio
    start=widgets.DatePicker(description='Start'),                       # typed manually
    end=widgets.DatePicker(description='End'),                           # typed manually
    trade_option=widgets.ToggleButtons(options=['Both', 'Buy', 'Sell'], description
)
```

```
HTML(value='<b>Data available:</b> Daily 03 Jan 2000 → 08 Jul 2025 | Weekly 06 Jan 2
000 → 08 Jul 2025')
interactive(children=(Dropdown(description='Time frame', options=('Daily', 'Weekl
y'), value='Daily'), Dropdown…
```

Out[61]:
```
<function __main__.run_dashboard(timeframe, symbol, strategy, account_size, risk_p
ct, rr_ratio, sl_atr_mult, start, end, trade_option)>
```